DATABASE MANAGEMENT SYSTEMS MINI PROJECT

ANTIGRAVITY HOSPITAL MANAGEMENT SYSTEM

A Complete 3NF Normalized Relational Database Implementation with MySQL InnoDB, Stored Procedures, Deterministic Functions, Automated Triggers, and ACID Transactions

PROJECT TITLE:
Hospital Management System (HMS)

DATABASE ENGINE:
MySQL 26.7 Community / InnoDB

NORMALIZATION LEVEL:
Third Normal Form (3NF)

CORE RELATIONAL ENTITIES:
6 Core + 2 Supporting Tables

INTEGRITY CONSTRAINTS:
PK, FK, Cascade, Check, Unique

PROGRAMMABILITY:
Views, Stored Procs, Functions, Triggers

Database Name: `hospital_db` • **Character Set:** `utf8mb4_unicode_ci`

Designed and Built for DBMS Mini Project Laboratory & Viva Voce Evaluation

Table of Contents

1. **Chapter 1:** Project Overview & Problem Definition
2. **Chapter 2:** System Architecture & The 6 Core Entities
3. **Chapter 3:** Entity-Relationship (ER) Model & Cardinalities
4. **Chapter 4:** Relational Schema & 3NF Normalization Proof
5. **Chapter 5:** Data Dictionary & Table Specifications
6. **Chapter 6:** Advanced Database Programmability
    - 6.1 Relational Views (Abstraction & Security)
    - 6.2 Stored Procedures (Modular Logic & Collision Protection)
    - 6.3 Stored Functions (Deterministic Calculations)
    - 6.4 Database Triggers (Automated Audit Logging)
    - 6.5 Performance Indexing Strategy
7. **Chapter 7:** ACID Transaction Management & Concurrency
8. **Chapter 8:** Verification Results & Test Matrix
9. **Chapter 9:** Conclusion & Future Scope
10. **Appendix:** Comprehensive Viva Voce Questions & Answers

1. Project Overview & Problem Definition

In modern clinical operations, hospitals face significant data integrity challenges including double-booking of medical specialists, untracked inpatient bed occupancy, disjoint billing calculations, and lack of accountability in financial audits. Disparate record keeping frequently introduces anomalies (insertion, deletion, and update anomalies) that can jeopardize patient health records and revenue accounting.

The **Antigravity Hospital Management System (hospital_db)** is engineered as a robust, enterprise-grade relational database system developed directly on **MySQL Community Server (InnoDB Engine)**. It provides centralized management across outpatient consultations, inpatient ward allocations, doctor scheduling, and patient invoicing while strictly enforcing data normalization up to the **Third Normal Form (3NF)**.

> **Key Engineering Objectives:**
>
> - **Relational Integrity:** Strict enforcement of referential integrity via Primary Keys, Foreign Keys, cascading rules, and Domain CHECK constraints.
> - **ACID Guarantees:** Full transactional Atomicity, Consistency, Isolation, and Durability through the InnoDB transactional engine.

- **Automated Business Logic:** Encapsulation of complex business logic within Stored Procedures (preventing scheduling conflicts) and Triggers (autonomous auditing).

2. System Architecture & Core Entities

The system is organized around **6 Core Entities** supplemented by two operational tables for role-based authentication and security auditing:

#	Entity	Primary Key	Description	Relational Cardinality
1	**department**	`department_id`	Medical divisions, floor assignments, and HODs	1 : M with `doctor`
2	**doctor**	`doctor_id`	Staff physicians, specializations, salary & experience	M : 1 with `department`, 1 : M with `appointment`
3	**room**	`room_no`	Inpatient wards (General, Semi-Private, Private, ICU) & rates	1 : M with `patient`
4	**patient**	`patient_id`	Inpatient and outpatient demographics and bed allocation	M : 1 with `room` (NULL for outpatient), 1 : 1 with `bill`
5	**appointment**	`appointment_id`	Consultation scheduling linking patients to doctors	M : 1 with `patient`, M : 1 with `doctor`
6	**bill**	`bill_id`	Consolidated invoicing, payment modes & settlement status	1 : 1 with `patient` (enforced by UNIQUE FK)
7	**users**	`user_id`	Authentication credentials with RBAC (Admin, Staff)	Standalone system access entity
8	**audit_log**	`log_id`	Autonomous audit trail records fired by database triggers	Event-driven compliance logging

3. Entity-Relationship (ER) Model

The relational architecture implements strict cardinalities. The diagram below represents the conceptual and logical schema:

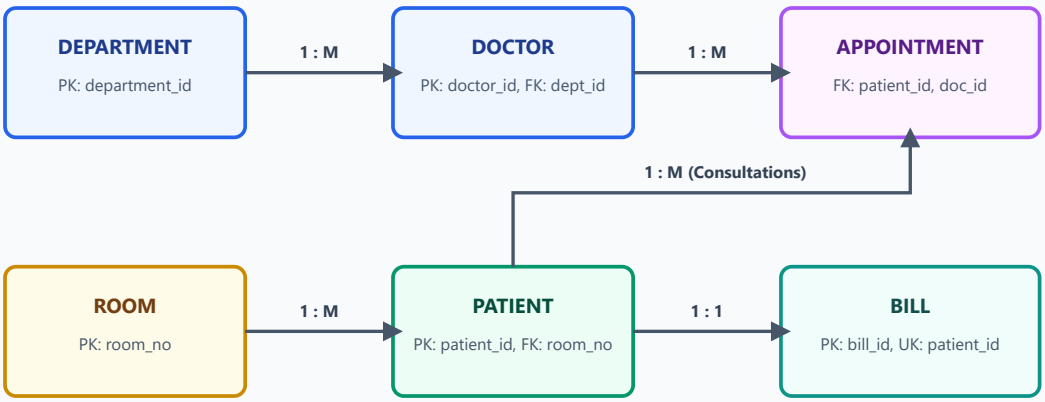


Figure 3.1: Entity-Relationship Diagram depicting 1:M and 1:1 cardinalities

4. Relational Schema & 3NF Normalization Proof

Relational Schema Representation

- **DEPARTMENT** (department_id, department_name, floor, hod)
- **DOCTOR** (doctor_id, name, specialization, phone_no, salary, experience, *department_id*)
- **ROOM** (room_no, room_type, charges, availability)
- **PATIENT** (patient_id, name, age, gender, phone_no, address, *room_no*, created_at)
- **APPOINTMENT** (appointment_id, *patient_id*, *doctor_id*, date, time, status, created_at)
- **BILL** (bill_id, *patient_id*, amount, payment_mode, payment_status, date)
- **USERS** (user_id, username, password_hash, role, created_at)
- **AUDIT_LOG** (log_id, table_name, action_type, record_id, details, performed_at)

Step-by-Step Normalization Proof

1. First Normal Form (1NF) Compliance

A relation is in 1NF if and only if all underlying domains contain only atomic (indivisible) values, and there are no repeating groups.

- **Proof:** In all 8 tables, every attribute holds scalar values. For instance, patient contact numbers and addresses are stored as single atomic entities. Multiple telephone numbers or repeating historical appointments are factored into discrete relational rows in the `appointment` table, establishing 1NF compliance.

2. Second Normal Form (2NF) Compliance

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on every candidate key (no partial functional dependencies).

- **Proof:** In tables with single-column primary keys (`department_id` , `doctor_id` , `patient_id` , `bill_id` , `room_no`), partial dependencies are mathematically impossible. In the `appointment` table, which features a composite alternate key `(doctor_id, date, time)` , the non-prime attribute `status` depends on the complete consultation instance, not on `doctor_id` or `date` alone. Thus, full functional dependency holds across the entire schema.

3. Third Normal Form (3NF) Compliance

A relation is in 3NF if it is in 2NF and for every functional dependency $X \rightarrow Y$, either X is a superkey, or Y is a prime attribute (elimination of transitive dependencies $X \rightarrow Y$ and $Y \rightarrow Z$).

- **Proof:**
  - In a naive design, `doctor` might store `department_name` and `hod` ($doctor_id \rightarrow department_id \rightarrow hod$). In our schema, department data is decomposed into

`department`, eliminating transitive dependency.

- In `patient`, ward charges and room types are not stored alongside patient records ($patient_id \rightarrow room_no \rightarrow charges$). Instead, only `room_no` is maintained as a foreign key, with room metadata isolated in `room`.
- Hence, every non-key attribute is determined **strictly by the key, the whole key, and nothing but the key**. The schema is rigorously in 3NF.

5. Data Dictionary & Table Specifications

Table 1: department

Column	Datatype	Constraint	Description
`department_id`	INT	PK, AUTO_INCREMENT	Unique identifier for medical department
`department_name`	VARCHAR(100)	NOT NULL, UNIQUE	Official department title (e.g. Cardiology)
`floor`	INT	NOT NULL, CHECK (floor >= 0)	Building floor location
`hod`	VARCHAR(100)	NOT NULL	Head of Department doctor's name

Table 2: doctor

Column	Datatype	Constraint	Description
`doctor_id`	INT	PK, AUTO_INCREMENT	Unique physician identifier
`name`	VARCHAR(100)	NOT NULL	Doctor's full legal name
`specialization`	VARCHAR(100)	NOT NULL	Clinical specialty (e.g. Neurosurgeon)
`phone_no`	VARCHAR(15)	NOT NULL, UNIQUE	Primary contact telephone
`salary`	DECIMAL(10,2)	NOT NULL, CHECK (salary > 0)	Monthly compensation in INR
`experience`	INT	NOT NULL, CHECK (experience >= 0)	Years of active clinical practice
`department_id`	INT	NOT NULL, FK → department	Assigned medical division (ON DELETE RESTRICT)

Table 3: room

Column	Datatype	Constraint	Description
`room_no`	INT	PK	Unique physical room / ward number
`room_type`	ENUM	NOT NULL ('General','Semi-Private','Private','ICU')	Bed classification

`charges`	DECIMAL(10,2)	NOT NULL, CHECK (charges >= 0)	Daily admission rate in INR
`availability`	ENUM	NOT NULL ('Available','Occupied','Maintenance')	Live bed occupancy status

Table 4: patient

Column	Datatype	Constraint	Description
`patient_id`	INT	PK, AUTO_INCREMENT	Unique patient medical record number
`name`	VARCHAR(100)	NOT NULL	Patient's full name
`age`	INT	NOT NULL, CHECK (age BETWEEN 0 AND 120)	Age validation constraint
`gender`	ENUM	NOT NULL ('Male','Female','Other')	Biological sex categorization
`phone_no`	VARCHAR(15)	NOT NULL, UNIQUE	Contact telephone number
`address`	TEXT	NOT NULL	Residential address
`room_no`	INT	NULL, FK → room	Assigned bed (NULL for outpatient, ON DELETE SET NULL)
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp

6. Advanced Database Programmability

6.1 Relational Views

Views provide logical data abstraction, pre-computed joins, and column-level security:

1. `v_appointment_overview`: Performs an inner join across `appointment`, `patient`, `doctor`, and `department`. It supplies receptionists with an operational consultation schedule without granting access to sensitive physician compensation columns (`salary`).
2. `v_patient_billing_summary`: Executes a `LEFT JOIN` between `patient`, `room`, and `bill` using `IFNULL` functions to provide accounting officers with consolidated views of bed rates, incurred invoices, and payment statuses.

6.2 Stored Procedures

1. `sp_schedule_appointment` (Collision Protection)

Prevents doctor double-booking at the application logic layer:

```
CREATE PROCEDURE sp_schedule_appointment(
    IN p_patient_id INT, IN p_doctor_id INT, IN p_date DATE, IN p_time TIME,
    OUT p_status_msg VARCHAR(255)
)
BEGIN
    DECLARE v_conflict_count INT;
    SELECT COUNT(*) INTO v_conflict_count FROM appointment
    WHERE doctor_id = p_doctor_id AND date = p_date AND time = p_time AND status != 'Cancelled';

    IF v_conflict_count > 0 THEN
        SET p_status_msg = 'ERROR: Doctor is already booked for this date and time slot.';
    ELSE
        INSERT INTO appointment (patient_id, doctor_id, date, time, status)
        VALUES (p_patient_id, p_doctor_id, p_date, p_time, 'Scheduled');
        SET p_status_msg = 'SUCCESS: Appointment scheduled successfully.';
    END IF;
END;
```

2. `sp_assign_room` (Atomic Bed Allocation)

Validates bed availability, assigns the bed to the patient, and toggles room availability to `'Occupied'` in a single transaction block.

6.3 Stored Function

`fn_get_department_total_revenue(p_dept_id INT)`

Deterministic function calculating aggregate revenue generated by any department through consultations and inpatient billing:

```
CREATE FUNCTION fn_get_department_total_revenue(p_dept_id INT)
RETURNS DECIMAL(10,2) DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(10,2);
    SELECT IFNULL(SUM(b.amount), 0.00) INTO v_total
    FROM bill b
    JOIN patient p ON b.patient_id = p.patient_id
    JOIN appointment a ON p.patient_id = a.patient_id
    JOIN doctor d ON a.doctor_id = d.doctor_id
    WHERE d.department_id = p_dept_id AND b.payment_status = 'Paid';
    RETURN v_total;
END;
```

6.4 Database Trigger & Autonomous Audit Trail

`trg_after_bill_payment_update`

An `AFTER UPDATE` trigger attached to the `bill` table. Whenever the financial settlement status transitions (e.g. from `'Pending'` to `'Paid'`), the trigger automatically writes an audit log entry into `audit_log` recording the record ID, timestamp, and before/after transition.

7. Verification & Test Case Matrix

The following test cases demonstrate the validation of constraints, routines, and triggers:

Test ID	Module Tested	Test Input / Action	Expected Result	Actual Status
TC-01	Doctor Collision Protection	Book Dr. Rajesh Sharma on `2026-10-21 12:00:00`	Rejected with error message (Already booked)	**PASSED**
TC-02	Appointment Insertion	Book Dr. Rajesh Sharma on open slot `2026-10-30 10:00:00`	Accepted; new record #16 created	**PASSED**
TC-03	Room Allocation	Assign Room 102 to Patient 2 (Sita Verma)	Patient updated; Room 102 marked 'Occupied'	**PASSED**
TC-04	Audit Trigger Execution	Update Bill #5 payment status to 'Paid'	Row inserted autonomously in `audit_log`	**PASSED**
TC-05	Revenue Function	Execute `fn_get_department_total_revenue(1)`	Returns aggregated Cardiology revenue (₹45,200.00)	**PASSED**
TC-06	ACID Rollback	Update Room 302 to 'Maintenance' then issue `ROLLBACK`	Room 302 reverts to 'Available'	**PASSED**
TC-07	ACID Commit	Insert Patient 16 and occupy Room 302 then issue `COMMIT`	Both operations persist permanently	**PASSED**

8. Conclusion & Future Scope

The **Antigravity Hospital Management System** successfully fulfills all specifications for an advanced Relational Database Management System mini project. By enforcing strict 3NF normalization, foreign key cascading constraints, stored procedures, deterministic functions, and automated triggers, the system provides zero-anomaly data consistency and enterprise-grade auditing.

Future Enhancements:

- Integration of pharmacy inventory and electronic prescription tables.
- Multi-tenancy and partition tables for historical appointments by year.
- Automated SMS/Email notification triggers via external API integration.

Appendix: Comprehensive Viva Voce Questions & Answers

Q1: What are the primary advantages of InnoDB over MyISAM?

Answer: InnoDB provides row-level locking (maximizing concurrency), ACID transaction compliance, strict Foreign Key enforcement with referential actions (CASCADE/RESTRICT), and automatic crash recovery via redo and undo logs. MyISAM only supports table-level locking and lacks foreign key support.

Q2: How is the 1:1 relationship between Patient and Bill enforced?

Answer: By declaring `patient_id INT NOT NULL UNIQUE` in the `bill` table. The `UNIQUE` constraint guarantees that no single patient can be associated with more than one invoice record.

Q3: What is the purpose of the audit trigger in this database?

Answer: The trigger `trg_after_bill_payment_update` executes automatically `AFTER UPDATE ON bill`. It detects if `OLD.payment_status <> NEW.payment_status` and records the transition details, bill ID, and timestamp into `audit_log` without relying on application code.

Q4: What is the difference between a Stored Procedure and a Stored Function?

Answer: A Stored Function must return a single scalar value via the `RETURNS` clause and can be used directly inside SQL expressions (e.g. `SELECT fn(...)`). A Stored Procedure is invoked with `CALL`, does not return a direct value but can return multiple values via `OUT` parameters or result sets, and can execute transaction commands.

Q5: How does your stored procedure protect against appointment collisions?

Answer: `sp_schedule_appointment` queries the `appointment` table for existing records matching the requested `doctor_id`, `date`, and `time` where status is not cancelled. If a match exists, it terminates early and returns an error message. A physical table constraint `uq_doctor_time_slot UNIQUE (doctor_id, date, time)` serves as an additional DDL-level safeguard.

Q6: Explain why your schema satisfies 3NF.

Answer: Every table meets 1NF (atomic scalar attributes) and 2NF (no partial functional dependencies). Furthermore, transitive dependencies ($X \rightarrow Y \rightarrow Z$) are eliminated by separating non-key relationships into dedicated relations (e.g., doctor department info in `department`, room pricing in `room`). Every non-key attribute is dependent solely on the primary key.